

Week 16: Angular Password Strength Evaluator - Report

Student Name: Enis Ogdu

Date: January 28, 2026

Course: Dynamic Documents and Front-end Web Development

Project Overview

This project implements a **Password Strength Evaluator** component using the Angular framework. The application provides real-time password strength evaluation with visual feedback, textual suggestions, and a show/hide password toggle feature.

Key Features

-  Real-time password strength evaluation
 -  Dynamic strength label with color-coded feedback (Weak/Fair/Good/Strong)
 -  Visual strength indicator bar
 -  Contextual suggestions for password improvement
 -  Show/hide password toggle functionality
 -  Modern, responsive UI with gradient styling
-

Technical Implementation

Architecture

The application follows Angular best practices with a component-service architecture:

- **Component:** PasswordEvaluatorComponent - Handles UI and user interactions
 - **Service:** PasswordEvaluator - Implements password strength evaluation logic
 - **Dependency Injection:** Service is injected into the component
-

Source Code

1. Password Evaluator Service

File: src/app/password-evaluator.ts

```
import { Injectable } from '@angular/core';

export interface PasswordStrength {
  score: number;           // 0-4 (Very Weak to Strong)
  label: string;           // "Weak", "Fair", "Good", "Strong"
```

```

    color: string;           // CSS color for visual feedback
    suggestions: string[];   // Array of improvement suggestions
}

@Inject({
  providedIn: 'root',
})
export class PasswordEvaluator {

  /**
   * Main method to evaluate password strength
   */
  evaluatePassword(password: string): PasswordStrength {
    if (!password || password.length === 0) {
      return {
        score: 0,
        label: 'Empty',
        color: '#6c757d',
        suggestions: ['Please enter a password']
      };
    }

    let score = 0;
    const suggestions: string[] = [];

    // Check Length
    const lengthScore = this.checkLength(password);
    score += lengthScore;

    if (password.length < 6) {
      suggestions.push('Password is too short (minimum 6 characters)');
    } else if (password.length < 9) {
      suggestions.push('Consider using a longer password (9+ characters)');
    }

    // Check character variety
    const varietyScore = this.checkCharacterVariety(password);
    score += varietyScore;

    const hasLower = /[a-z]/.test(password);
    const hasUpper = /[A-Z]/.test(password);
    const hasNumber = /[0-9]/.test(password);
    const hasSymbol = /^[^a-zA-Z0-9]/.test(password);

    if (!hasSymbol) {
      suggestions.push('Add special characters (!@#$%^&*)');
    }
    if (!hasNumber) {
      suggestions.push('Add numbers');
    }
  }
}

```

```

}
if (!hasUpper) {
  suggestions.push('Add uppercase letters');
}
if (!hasLower) {
  suggestions.push('Add lowercase letters');
}

// Check for repeated characters
if (this.hasRepeatedCharacters(password)) {
  score -= 1;
  suggestions.push('Avoid repeated characters');
}

// Check for sequential characters
if (this.hasSequentialCharacters(password)) {
  score -= 0.5;
  suggestions.push('Avoid sequential characters (e.g., abc, 123)');
}

// Normalize score to 0-4 range
score = Math.max(0, Math.min(4, score));

// Determine label and color based on score
let label: string;
let color: string;

if (score <= 1) {
  label = 'Weak';
  color = '#dc3545'; // Red
} else if (score <= 2) {
  label = 'Fair';
  color = '#fd7e14'; // Orange
} else if (score <= 3) {
  label = 'Good';
  color = '#ffc107'; // Yellow
} else {
  label = 'Strong';
  color = '#28a745'; // Green
  if (suggestions.length === 0) {
    suggestions.push('Excellent password!');
  }
}

return {
  score,
  label,
  color,
  suggestions
}

```

```

    };
}

/**
 * Check password length and return score
 */
private checkLength(password: string): number {
    if (password.length < 6) return 0;
    if (password.length < 9) return 1;
    if (password.length < 13) return 1.5;
    return 2;
}

/**
 * Check character variety and return score
 */
private checkCharacterVariety(password: string): number {
    let varietyScore = 0;

    if (/[a-z]/.test(password)) varietyScore += 0.5;
    if (/[A-Z]/.test(password)) varietyScore += 0.5;
    if (/[\d]/.test(password)) varietyScore += 0.5;
    if (/^[a-zA-Z\d]/.test(password)) varietyScore += 1;

    return varietyScore;
}

/**
 * Check for repeated consecutive characters
 */
private hasRepeatedCharacters(password: string): boolean {
    return /(.)\1/.test(password);
}

/**
 * Check for sequential characters
 */
private hasSequentialCharacters(password: string): boolean {
    const sequences = ['abc', 'bcd', 'cde', 'def', 'efg', 'fgh', 'ghi',
'hij',
                        'ijk', 'jkl', 'klm', 'lmn', 'mno', 'nop', 'opq',
'pqr',
                        'qrs', 'rst', 'stu', 'tuv', 'uvw', 'vwx', 'wxy',
'xyz',
                        '012', '123', '234', '345', '456', '567', '678',
'789'];

    const lowerPassword = password.toLowerCase();
    return sequences.some(seq => lowerPassword.includes(seq));
}

```

```
}  
}
```

2. Password Evaluator Component

File: src/app/password-evaluator/password-evaluator.ts

```
import { Component } from '@angular/core';  
import { FormsModule } from '@angular/forms';  
import { CommonModule } from '@angular/common';  
import { PasswordEvaluator, PasswordStrength } from '../password-evaluator';  
  
@Component({  
  selector: 'app-password-evaluator',  
  imports: [FormsModule, CommonModule],  
  templateUrl: './password-evaluator.html',  
  styleUrls: ['./password-evaluator.css'],  
})  
export class PasswordEvaluatorComponent {  
  password: string = '';  
  showPassword: boolean = false;  
  strengthInfo: PasswordStrength | null = null;  
  
  constructor(private passwordEvaluator: PasswordEvaluator) {}  
  
  /**  
   * Toggle password visibility  
   */  
  togglePasswordVisibility(): void {  
    this.showPassword = !this.showPassword;  
  }  
  
  /**  
   * Handle password input changes and evaluate strength  
   */  
  onPasswordChange(): void {  
    this.strengthInfo =  
this.passwordEvaluator.evaluatePassword(this.password);  
  }  
}
```

File: src/app/password-evaluator/password-evaluator.html

```
<div class="password-evaluator-container">  
  <h2>Password Strength Evaluator</h2>  
  
  <div class="password-input-wrapper">  
    <label for="password-input">Enter Password:</label>
```

```

<div class="input-group">
  <input
    id="password-input"
    [type]="showPassword ? 'text' : 'password'"
    [(ngModel)]="password"
    (input)="onPasswordChange()"
    placeholder="Type your password..."
    class="password-input"
  />
  <button
    type="button"
    class="toggle-button"
    (click)="togglePasswordVisibility()"
    [attr.aria-label]="showPassword ? 'Hide password' : 'Show password'"
  >
    <span class="eye-icon">{{ showPassword ? '👁' : '👁' }}</span>
  </button>
</div>
</div>

<div class="strength-indicator" *ngIf="strengthInfo && password.length >
0">
  <div class="strength-label-wrapper">
    <span class="strength-label-text">Strength:</span>
    <span
      class="strength-label"
      [style.color]="strengthInfo.color"
      [style.border-color]="strengthInfo.color"
    >
      {{ strengthInfo.label }}
    </span>
  </div>

  <div class="strength-bar-container">
    <div
      class="strength-bar"
      [style.width]="(strengthInfo.score / 4 * 100) + '%"
      [style.background-color]="strengthInfo.color"
    ></div>
  </div>

  <div class="suggestions" *ngIf="strengthInfo.suggestions.length > 0">
    <h4>Suggestions:</h4>
    <ul>
      <li *ngFor="let suggestion of strengthInfo.suggestions">{{ suggestion
}}</li>
    </ul>
  </div>

```

```
</div>
</div>
```

File: src/app/password-evaluator/password-evaluator.css

```
.password-evaluator-container {
  max-width: 500px;
  margin: 40px auto;
  padding: 30px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border-radius: 20px;
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
}

h2 {
  color: white;
  text-align: center;
  margin-bottom: 30px;
  font-size: 28px;
  font-weight: 600;
  text-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.password-input-wrapper {
  margin-bottom: 25px;
}

label {
  display: block;
  color: white;
  font-weight: 500;
  margin-bottom: 10px;
  font-size: 16px;
}

.input-group {
  position: relative;
  display: flex;
  align-items: center;
}

.password-input {
  flex: 1;
  padding: 14px 50px 14px 16px;
  font-size: 16px;
  border: 2px solid rgba(255, 255, 255, 0.3);
  border-radius: 12px;
  background: rgba(255, 255, 255, 0.95);
  transition: all 0.3s ease;
  outline: none;
}
```

```

}

.password-input:focus {
  border-color: rgba(255, 255, 255, 0.8);
  background: white;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

.toggle-button {
  position: absolute;
  right: 8px;
  background: transparent;
  border: none;
  cursor: pointer;
  padding: 8px;
  display: flex;
  align-items: center;
  justify-content: center;
  transition: transform 0.2s ease;
}

.toggle-button:hover {
  transform: scale(1.1);
}

.toggle-button:active {
  transform: scale(0.95);
}

.eye-icon {
  font-size: 24px;
  user-select: none;
}

.strength-indicator {
  background: rgba(255, 255, 255, 0.95);
  padding: 20px;
  border-radius: 12px;
  margin-top: 20px;
  animation: fadeIn 0.3s ease;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(-10px);
  }
  to {
    opacity: 1;
  }
}

```



```

        transform: translateY(0);
    }
}

.strength-label-wrapper {
    display: flex;
    align-items: center;
    gap: 12px;
    margin-bottom: 15px;
}

.strength-label-text {
    font-weight: 600;
    color: #333;
    font-size: 16px;
}

.strength-label {
    padding: 6px 16px;
    border-radius: 20px;
    font-weight: 700;
    font-size: 14px;
    border: 2px solid;
    background: rgba(255, 255, 255, 0.5);
    transition: all 0.3s ease;
}

.strength-bar-container {
    width: 100%;
    height: 12px;
    background: #e0e0e0;
    border-radius: 6px;
    overflow: hidden;
    margin-bottom: 20px;
}

.strength-bar {
    height: 100%;
    transition: width 0.4s ease, background-color 0.4s ease;
    border-radius: 6px;
}

.suggestions {
    margin-top: 15px;
}

.suggestions h4 {
    color: #333;
    font-size: 16px;
}

```

```
    margin-bottom: 10px;
    font-weight: 600;
}

.suggestions ul {
    list-style: none;
    padding: 0;
    margin: 0;
}

.suggestions li {
    padding: 8px 12px;
    margin-bottom: 6px;
    background: #f8f9fa;
    border-left: 3px solid #667eea;
    border-radius: 4px;
    color: #555;
    font-size: 14px;
    transition: all 0.2s ease;
}

.suggestions li:~hover {
    background: #e9ecef;
    transform: translateX(4px);
}

.suggestions li:~last-child {
    margin-bottom: 0;
}
```

Functionality Demonstrations

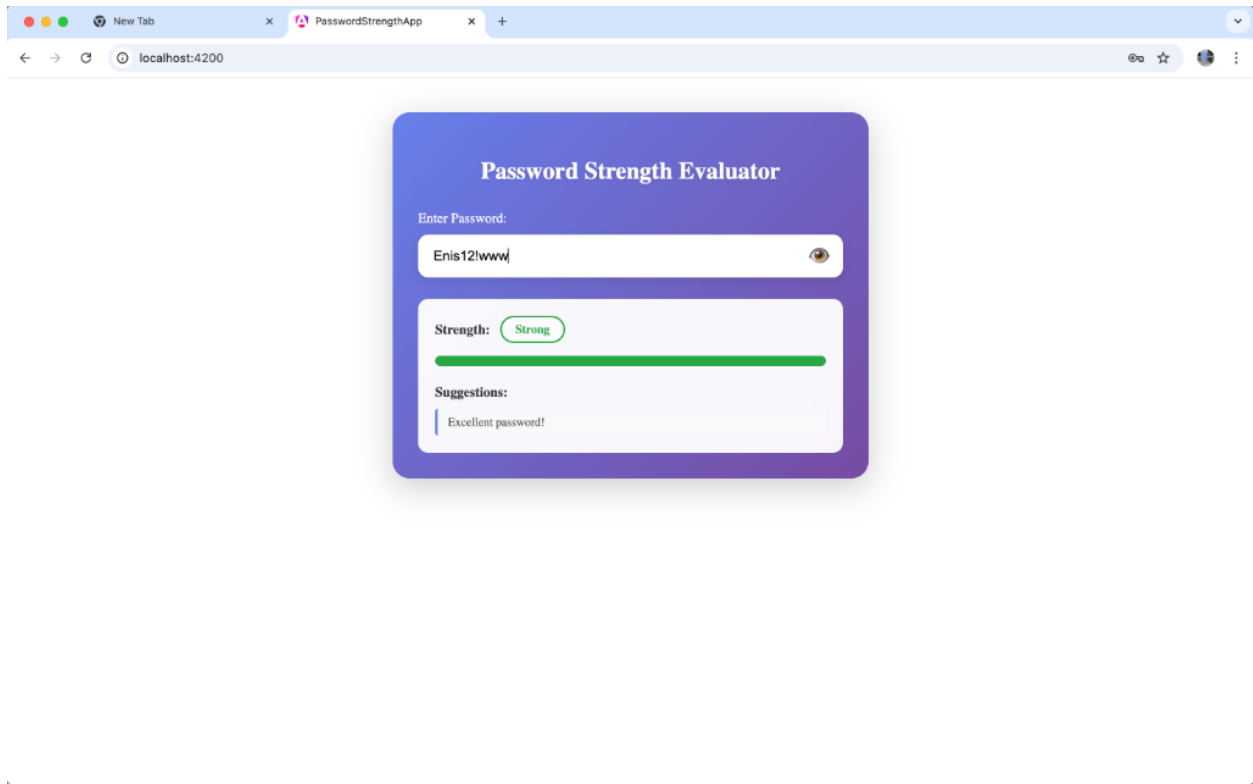
1. Strong Password Example

Password: Enis12!www

Strong Password

Strong Password

Features Demonstrated: -  Green “Strong” label indicating excellent password strength
-  Full strength bar (100%) -  Positive feedback: “Excellent password!” -  Password is visible (show/hide toggle active)



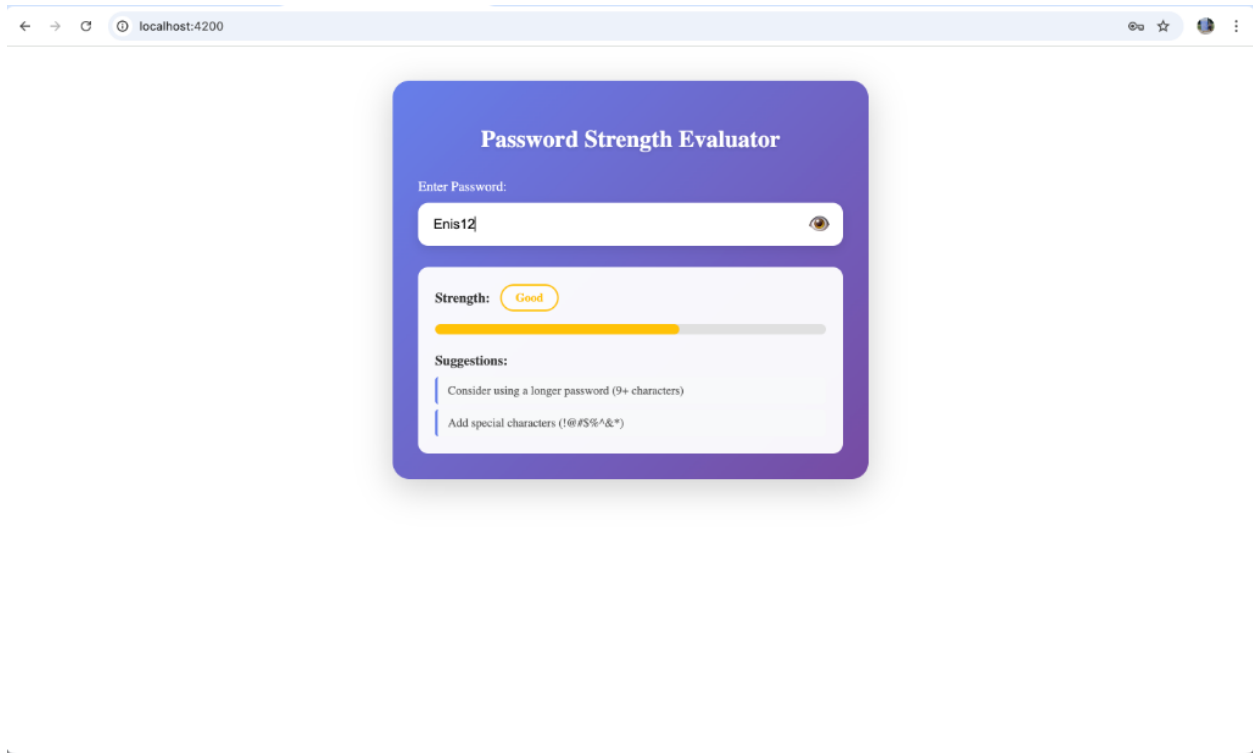
2. Good Password Example

Password: Enis12

Good Password

Good Password

Features Demonstrated: - ☒ Yellow “Good” label indicating good password strength - ☒ Partial strength bar (~60%) - ☒ Helpful suggestions: - “Consider using a longer password (9+ characters)” - “Add special characters (!@#\$\$%^&*)”



3. Weak Password Example

Password: •••• (hidden password)

Weak Password

Weak Password

Features Demonstrated: - ☒ Red “Weak” label indicating poor password strength - ☒
Minimal strength bar (~25%) - ☒ Password is hidden (show/hide toggle inactive) - ☒
Multiple improvement suggestions: - “Password is too short (minimum 6 characters)” -
“Add special characters (!@#\$%^&*)” - “Add numbers”



Password Strength Evaluator

Enter Password:

Strength: Weak

Suggestions:

- Password is too short (minimum 6 characters)
- Add special characters (!@#\$%^&*)
- Add numbers

Password Strength Evaluation Logic

Scoring System

The password strength is evaluated based on multiple criteria:

- Length Check** (0-2 points)
 - < 6 characters: 0 points
 - 6-8 characters: 1 point
 - 9-12 characters: 1.5 points
 - 13+ characters: 2 points
- Character Variety** (0-2.5 points)
 - Lowercase letters: +0.5 points
 - Uppercase letters: +0.5 points
 - Numbers: +0.5 points
 - Special characters: +1 point
- Penalties**
 - Repeated characters: -1 point
 - Sequential characters (abc, 123): -0.5 points

Strength Categories

Score Range	Label	Color	Description
0-1	Weak	Red (#dc3545)	Poor password, needs significant improvement
1.1-2	Fair	Orange (#fd7e14)	Acceptable but could be stronger
2.1-3	Good	Yellow (#ffc107)	Good password with minor improvements possible
3.1-4	Strong	Green (#28a745)	Excellent password meeting all criteria

Technologies Used

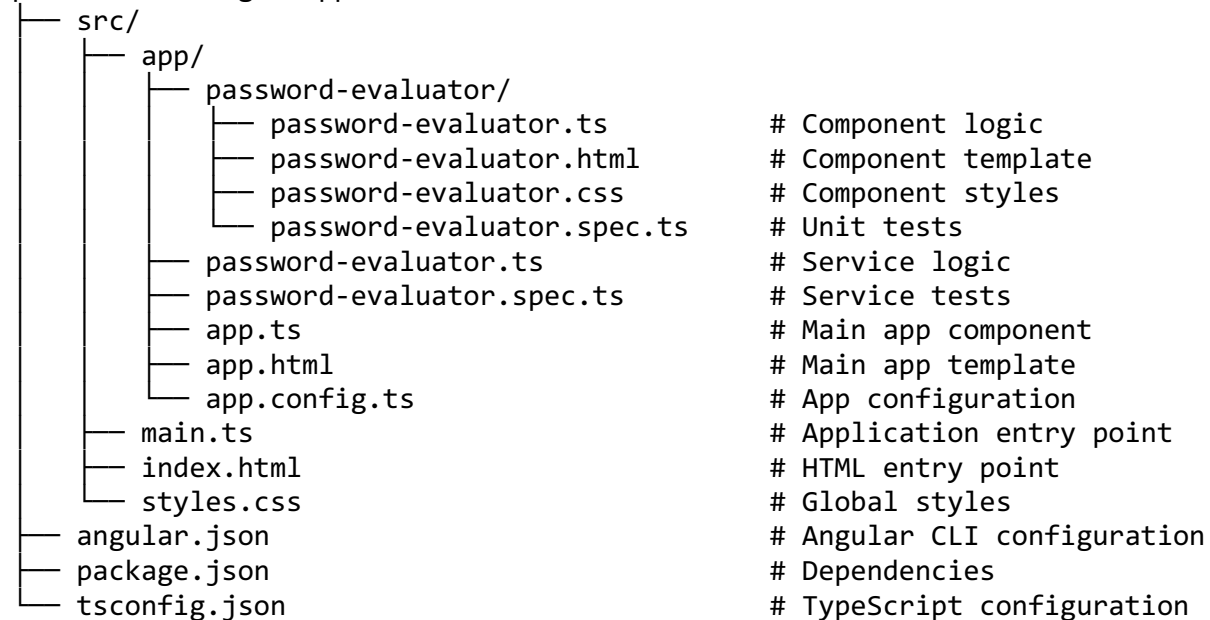
- **Angular 19** - Modern web framework
- **TypeScript** - Type-safe programming language
- **RxJS** - Reactive programming (built into Angular)
- **FormsModule** - Two-way data binding with [(ngModel)]
- **CommonModule** - Angular directives (*ngIf, *ngFor)
- **CSS3** - Modern styling with gradients and animations

Key Learning Outcomes

1.  **Angular Component Architecture** - Created standalone components with proper separation of concerns
2.  **Dependency Injection** - Injected service into component for business logic
3.  **Two-way Data Binding** - Used [(ngModel)] for reactive password input
4.  **Template Syntax** - Implemented Angular directives and property binding
5.  **TypeScript Interfaces** - Defined type-safe data structures
6.  **Regular Expressions** - Used regex patterns for password validation
7.  **CSS Animations** - Created smooth transitions and visual feedback
8.  **Responsive Design** - Built mobile-friendly UI with modern aesthetics

Project Structure

password-strength-app/



Running the Application

Development Server

```
cd password-strength-app
npx ng serve
```

Navigate to <http://localhost:4200/> in your browser.

Production Build

```
npx ng build --configuration production
```

The build artifacts will be stored in the `dist/` directory.

Conclusion

This project successfully demonstrates the implementation of a fully functional password strength evaluator using Angular. The application provides real-time feedback with an intuitive user interface, helping users create stronger passwords through visual cues and actionable suggestions.

The implementation showcases modern Angular development practices including: - Standalone components - Service-based architecture - Reactive forms - Type-safe TypeScript code - Modern CSS with animations - Accessibility features

The password evaluation logic effectively identifies common password weaknesses and provides specific guidance for improvement, making it a practical tool for enhancing password security.

End of Report